Day 1 — Database Structures & Data Logic

> **Activity packet** for participant triads. Today's job: model the **entities** the feature reads, writes, and indexes; defend the **storage choice**; and draft TMD Section 1.

Where we are in the week

The TCD's component diagram showed *that* there's a datastore. This week's work names *what's in it* and *how it's shaped*. By 16:00, every triad has TMD Section 1 — a data model an engineer could begin coding against.

Inputs

- TCD Section 2 (component diagram + integrations)
- TCD Section 3 (security/compliance — informs encryption + key handling at rest)
- TCD Section 4 (SLOs — latency targets affect index choices and read-vs-write split)
- PRD Section 6 AC and Section 7 NFRs (data-shape implications)

The vocabulary card (today's reference)

Term	What it means	TPM-relevant signal
Entity	A "thing" the system tracks (Dispatcher, Ticket, Reconcile)	The unit of the data model
Primary key	The unique identifier for an entity	Affects how entities are referenced
Foreign key	A reference from one entity to another	Models relationships
Index	A structure that speeds up specific reads at the cost of writes and storage	Tied directly to query patterns
Normalization	Splitting data into multiple entities to avoid duplication	Cleaner; sometimes slower
Denormalization	Duplicating data on purpose for read speed	Faster reads; harder consistency
Constraint	A rule the database enforces (NOT NULL, UNIQUE, CHECK)	Catches bugs before they ship
Cardinality	How many of one thing relate to how many of another (1:1, 1:N, N:N)	Governs the relationship shape
Schema	The set of definitions for entities + relationships + constraints	The artifact we ship today

Storage choice — the four buckets

For most B2B features, the storage decision is one of:

Bucket	When to reach for it
Relational (Postgres, MySQL)	Strong relationships, transactions, complex queries; default for most B2B
Document (MongoDB, Firestore)	Flexible schema, hierarchical data; good when shape varies per record
Key-value (Redis, Cosmos DB)	High throughput, simple lookups, caches, session state
Search (Elasticsearch, OpenSearch)	Full-text search, analytics on text

A TPM doesn't pick the engine. The TPM **describes the access pattern** clearly enough that the architect can pick.

The "access pattern first" discipline

The most common rookie mistake: design the schema before writing the queries. The mature practice:

1. **List the queries the feature needs.** "Show me a dispatcher's open tickets, sorted by latest activity." "Mark all selected tickets as reconciled."
2. **For each query: what does it read, in what order, with what filter?**
3. **Now design the schema** so the queries are cheap.

If your data model can't be defended *by the queries that drive it*, it's wrong.

Activity 1 — Read Patterns + Write Patterns

Format: Triad • **35 min** • Block 1

Purpose

Surface every read and write the feature needs *before* drawing the schema.

Setup

Each triad needs the PRD Section 5 sketch, PRD Section 6 AC list, and the Access Pattern Sheet template. AI optional.

Triad protocol

1. **Re-read the PRD Section 5 sketch** (5 min). What does the user do? What screens load? What submits send?
2. **List read patterns** (15 min). For each screen / API call:
    - What data is shown?
    - What filters / sorts / pagination apply?
    - How fresh must it be? (real-time, last-minute, last-hour)
3. **List write patterns** (10 min). For each user action that modifies data:
    - What entity is created / updated / deleted?
    - What invariants must hold? ("a ticket can only be reconciled once")
    - What concurrency conflicts are possible?
4. **Highlight the high-volume reads** (5 min). Which 1–2 reads will run **most often**? Those drive index choices.

Output

A two-column **Access Pattern Sheet**:

#	Pattern	Type (R/W)	Data	Frequency	Freshn req.
1	List dispatcher's open tickets	R	tickets WHERE dispatcher_id = ? AND status = open	every screen load	last-mir
2	Mark tickets reconciled	W	update tickets, insert reconcile_event	once per shift per dispatcher	n/a
3	Audit-trail query for compliance	R	reconcile_events WHERE dispatcher_id = ? AND ts BETWEEN ? AND ?	rare	n/a
4	Dispatcher's reconcile history (last 7d)	R	reconcile_events WHERE dispatcher_id = ? ORDER BY ts DESC LIMIT 30	every reconcile screen	last-hou

---

Activity 2 — Draft the Entity Model

Format: Triad • **40 min** • Block 2

Purpose

Convert the access patterns into an entity model. Tables / collections, keys, relationships, indexes.

Setup

Each triad needs the Access Pattern Sheet from Activity 1 and the entity-model template. AI optional.

The entity model template

For each entity, capture:

```
## Entity: <Name>

| Field | Type | Notes |
|-------|------|-------|
| id    | UUID | Primary key |
| ...   | ...  | NOT NULL? UNIQUE? FK to ...? |

**Indexes:**
- (field, field) — purpose: query #N from access
patterns

**Relationships:**
- belongs_to <Entity> via <fk_field>
- has_many <Entity> via <fk_field>

**Notes / invariants:**
- A reconcile_event is immutable once created.
- ticket.status is one of {open, reconciled,
voided}.
```

Triad protocol

1. **List the entities** (10 min). Include both new entities (you're creating) and existing entities you're reading from.
2. **For each new entity, draft the fields** (15 min). Use the template.
3. **Mark the indexes** (10 min). For each entity, what indexes serve the high-frequency queries from Activity 1?
4. **Mark relationships and cardinalities** (5 min). 1:1, 1:N, N:N — diagram them or list them.

What "good" looks like

- Each new entity has a **primary key** stated
- Indexes reference **specific access patterns** by number
- **Cardinalities are explicit** — "a Dispatcher has many ReconcileEvents; a ReconcileEvent has one Dispatcher"
- No surprise columns — every field has a query that uses it

Worked example — FieldPulse reconcile

```
## Entity: ReconcileEvent

| Field | Type | Notes |
|-------|------|-------|
| id | UUID | PK |
| dispatcher_id | UUID | FK → Dispatcher |
| ticket_ids | UUID[] | The set reconciled (N:N
modeled inline; see notes) |
| submitted_at | timestamp | NOT NULL |
| client_ip | inet | for audit |
| user_agent | text | for audit |
| signature_hash | text | for non-repudiation (R-
threat from STRIDE) |

**Indexes:**
- (dispatcher_id, submitted_at DESC) — serves access
pattern #4 (history)
- (submitted_at) — serves access pattern #3 (audit-
trail range query)

**Relationships:**
- belongs_to Dispatcher via dispatcher_id

**Notes / invariants:**
- Immutable: never UPDATE'd after INSERT
- ticket_ids array is a denormalization (faster
read; see Activity 3 trade-off)
```

Deliverable

Entity-model draft with PKs, indexes referencing access patterns by number, named cardinalities, and explicit invariants.

Activity 3 — Storage Trade-Offs

Format: Triad • **40 min** • Block 3

Purpose

For each non-trivial design choice, surface the trade-off explicitly. Today's choices are local — schema-level — but the discipline is the same as Week 4's architectural trade-offs.

Setup

Each triad needs the entity model from Activity 2, TCD Section 4 SLOs for cross-reference, and the Week-4 trade-off template.

Three trade-offs every model encounters

Trade-off	Tension
Normalization vs denormalization	Clean writes vs fast reads
Strong consistency vs replica reads	Always-fresh vs scalable read load
Schema flexibility vs schema integrity	Easy migration vs caught-by-database integrity

Worked example: ticket_ids array vs join table

```
### Trade-off — ticket_ids storage

**Tension:** Normalization vs denormalization.

**Option A:** ReconcileEvent has ticket_ids: UUID[]
(denormalized).
**Option B:** Separate ReconcileEventTicket join
table (normalized).

**Choice:** Option A.

**Why:** Reconcile events are read in batches of 1
(a single event)
        most of the time. Reading the array of IDs
is one row. Option B
        requires a join. Reconcile events are
immutable and rarely
        queried by ticket — the access patterns
don't reward normalization.

**Accepted cost:** Cannot index by individual
ticket_id (no
        "find every reconcile that touched ticket X"
query). For
        compliance queries that need this, we'd add
a separate
        TicketReconcileHistory denormalization.

**Revisit trigger:** If "find reconciles touching
ticket X" becomes
        a high-volume query.
```

Triad protocol

1. **Identify 3 trade-off points** in your model (15 min). For each: what was the choice, what was the alternative?
2. **Use the Week-4 trade-off template** (15 min). Same Option A/B/Choice/Why/Cost/Trigger.
3. **Cross-check against TCD Section 4 SLOs** (10 min). Does any choice push against a latency target? If yes, name it.

What "good" looks like

- At least one trade-off references a **TCD SLO**

- "Accepted cost" is a query you **deliberately can't run efficiently** today
- "Revisit trigger" is a query frequency change

Deliverable

3 schema-level trade-offs using the Week-4 template, at least one cross-referencing a TCD Section 4 SLO.

Activity 4 — AI-Assisted Schema Critique + Polish

Format: Triad • **45 min** + Wrap • Block 4

Purpose

Use AI as a critic. Update Section 1. Add the provenance log entry.

Setup

Each triad needs the entity model, the Access Pattern Sheet, and the trade-off list. AI required; log provenance.

The two prompts

A. "Critique this schema"

```
Role: Senior backend engineer reviewing a feature's
data model.
Context: <paste entity definitions + indexes +
relationships +
         the access pattern sheet from Activity 1>
Task: Identify the top 3 issues.
Constraints:
  - For each issue, name the specific access pattern
or invariant
    that breaks
  - Suggest a concrete fix (new index, new entity,
or different
    cardinality)
  - Do not suggest generic 'best practices'
Format: Numbered list — Issue / What breaks / Fix.
```

B. "What did we forget?"

```
Continuing from above. Given these access patterns,
what *queries
the user will eventually want* are not yet supported
by this
schema? List 3, ranked by likelihood of mattering
within 6 months.
```

Triad protocol

1. **Run Prompt A** (10 min). Adopt / defer / reject each issue.
2. **Run Prompt B** (10 min). Decide which deferred queries to capture in Section 11 (out-of-scope follow-ups).
3. **Update Section 1** (15 min). Final entity model + indexes + trade-offs.
4. **Provenance note** (5 min). What prompts, what was adopted/rejected.
5. **AI-prose check** (5 min). Rewrite anything generic in your own voice.

Deliverable

Polished TMD Section 1 (entity model + trade-offs) incorporating adopted AI findings, plus a provenance note logging prompts, adoptions, and rejections.

Wrap (last 15 min)

Each triad shares:

- One **trade-off** they made explicit (with revisit trigger)
- One **query** AI surfaced that would have caught them in 6 months
- One **question for the architect** about storage choice

End-of-day checkpoint

Each triad ends Day 1 with:

- [x] **Access Pattern Sheet** (read + write patterns, with frequency + freshness)
- [x] **Entity model** with PKs, indexes, relationships, invariants
- [x] At least 3 **explicit trade-offs** in the model

- [x] **Provenance log** entry for AI prompts used today
- [x] TMD Section 1 drafted